

Linux文本处理命令和服务管理

一、Linux文本处理命令



(1.1) 字符串过滤

1.1.1 grep命令

grep在基础命令的时候已经介绍过，主要是从文本内容中过滤包含关键词的行（**输出结果是整行信息**）

比如，查看shadow文件中，和root信息相关的行信息

```
grep root /etc/shadow
```

```
[root@theon ~]# grep root /etc/shadow
root:$6$eVXgAfZQ$aEamkzzVYJLkC0yFHLu1vQZwhFy10oFgmH3h7CMLmsrI4qKrZYPPXidWGBjmvwoq1rqb6ScsiLHDLNmdLYp/:20201:0:99999:7:::
```

通常，grep是和管道符|结合进行使用

```
cat /etc/shadow|grep root
```

(1.2) 字符串截取

结合1.1.1 中grep命令的例子，如果假设我不需要grep输出的整行信息，而是只需要行中的某列，比如我只需要1.1.1例子grep输出的行信息中，root的密文密码信息。应该如何实现？

可以使用linux的字符串截取命令-----从行中截取列信息的命令（输出结果是某列或多列）

1.2.1 cut命令

命令格式

```
cut [选项] 文件名（文本内容）
```

选项：

-f 列号：提取第几列

-d分隔符：按照指定分隔符分隔列

cut 命令默认分隔符为“tab”制表符

备注：cut由于处理的也是文本内容，所以也可以和管道符|结合进行使用

范例，截取root的密文密码信息

```
cat /etc/shadow|grep root | cut -d ":" -f 2  
# cut -d ":" -f 2 /etc/shadow # 这个输出的是什么？
```

```
[root@theon ~]# cat /etc/shadow|grep root | cut -d ":" -f 2  
$6$eVXgAfZQ$aEamkzzVYJLkC0yfHLu1vQZwhFyi0oFgMmH3h7CMLCmsrI4qKrZYPPXidWGBjmvwoq1rqbGScsiLHDLNmdLYp/  
[root@theon ~]#
```

1.2.2 awk命令

awk是Linux/Unix系统中一个非常强大的文本处理工具，它不仅仅是一个命令，更是一种编程语言

命令格式

```
awk '条件1{动作1} 条件2{动作2} 条件3{动作3}...'
```

条件

一般使用关系表达式作为条件，比如

x>10 判断变量 x是否大于10

x>=10 大于等于

<=10 小于等于

条件可以省略

awk最常用的动作就是输出，使用print、printf命令进行输出

动作

格式化输出

流程控制语句

awk默认识别分隔符为：制表符 空格

awk也可以和管道符 | 结合进行使用

awk的输出动作使用print或者printf实现

范例，查看磁盘sda的使用量

```
df -h|grep sda|awk '{print $3}'
```

范例，查看shadow文件中，root的密文密码

```
cat /etc/shadow|grep root|awk -F":" '{print $2}'
```

- **AWK的特定条件：BEGIN**

表示在动作之前优先执行的操作

如下范例

```
cat /etc/shadow|grep root|awk -F":" 'BEGIN{printf"root的密码是："}{print $2}'
```

表示在输出列这个动作之前，先printf打印一段字符串

```
[root@theon ~]# cat /etc/shadow|grep root|awk -F":" 'BEGIN{printf"root的密码是："}{print $2}'
root的密码是：$6$eVXgAfZQ$aEamkzzVYJLkCOyfHLu1vQZwhFyi0oFgMmH3h7CMLCmsrI4qKrZYPPXidWGBjmvwoq1rqbGScsiLHDLNmdLYp/
[root@theon ~]#
```

- **AWK的特定条件：END**

表示在动作之后才执行的操作

如下范例

```
cat /etc/shadow|grep root|awk -F":" 'END{print" →→→这个是root的密码"}{printf $2}'
```

```
[root@theon ~]# cat /etc/shadow|grep root|awk -F":" 'END{print" →→→这个是root的密码"}{printf $2}'
$6$eVXgAfZQ$aEamkzzVYJLkCOyfHLu1vQZwhFyi0oFgMmH3h7CMLCmsrI4qKrZYPPXidWGBjmvwoq1rqbGScsiLHDLNmdLYp/ →→→这个是root的密码
[root@theon ~]#
```

- **FS内置变量**

和 -F 参数功能一样，表示识别什么为分隔符,通常和BEGIN组合使用

如下范例

```
#cat /etc/shadow|grep root|awk -F":" 'BEGIN{printf"root的密码是："}{print $2}'
cat /etc/shadow|grep root|awk 'BEGIN{FS=":"}{printf"root的密码是："}{print $2}'
```

- **AWK条件判断**

AWK可以使用一些简单的条件判断来选择输出符合条件的列

如下范例，功能是输出UID大于等于1000的用户，即输出所有普通用户

```
# $3>=1000{print $1 表示，如果第三列大于1000条件成立，那么就输出第一列
cat /etc/passwd|awk -F":" '$3>=1000{print $1}'
#输出uid为0的用户，即超级管理员用户
#cat /etc/passwd|awk -F":" '$3==0{print $1}'
```

(1.3) 字符串替换

1.3.1 sed命令

sed 是一种几乎包括在所有 UNIX平台(包括 Linux)的轻量级流编辑器。sed主要是用来将数据进行选取、替换、删除、新增的命令。

如果我们在shell脚本中需要修改一些配置文件内容时，由于无法使用vim命令，就会使用sed命令来无交互的修改配置文件内容

命令格式

```
sed [选项]'[动作]' 文件名（文本内容）
```

选项

-n: 一般sed命令会把所有数据都输出到屏幕如果加入此选择，则只会把经过sed命令处理的行输出到屏幕。

-e: 允许对输入数据应用多条sed命令编辑

-i: sed的修改结果直接修改读取数据的文件而不是由屏幕输出

动作

a: 追加，在当前行后面添加一行或多行。添加多行时，除最后一行外，每行末尾需要用“\”代表数据未完结。以被重定向替代

c: 行替换，用c后面的字符串替换原数据行，替换多行时，除最后一行外，每行末尾需用“\”代表数据未完结。

i: 插入，在当期行前插入一行或多行。插入多行时，除最后一行外，每行末尾需要用“\”代表数据未完结。

d: 删除，删除指定的行。

p: 打印，输出指定的行。

s: 字符串替换，用一个字符串替换另外一个字符串。格式为“行范围s/旧字符串/新字符串/g”(和vim中的替换格式类似)

数据操作范例

- 查看文件第一行

通常会添加-n参数，否则会输出文件所有内容

```
sed "1p" /etc/passwd
sed -n "1p" /etc/passwd
```

```
[root@theon ~]# sed "1p" /etc/passwd
root:x:0:0:root:/root:/bin/bash
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
operator:x:11:0:operator:/root:/sbin/nologin
games:x:12:100:games:/usr/games:/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
nobody:x:99:99:Nobody:/:/sbin/nologin
systemd-network:x:192:192:systemd Network Management:/:/sbin/nologin
dbus:x:81:81:System message bus:/:/sbin/nologin
polkitd:x:999:998:User for polkitd:/:/sbin/nologin
sshd:x:74:74:Privilege-separated SSH:/var/empty/sshd:/sbin/nologin
postfix:x:89:89:/:/var/spool/postfix:/sbin/nologin
wsder:x:1000:1000:/:/home/wsder:/bin/bash
nginx:x:998:996:Nginx web server:/var/lib/nginx:/sbin/nologin
theon:x:1001:1001:/:/home/theon:/bin/bash
abc:x:1002:1002:/:/home/abc:/bin/bash
bike:x:1003:1003:/:/home/bike:/bin/bash
tcpdump:x:72:72:/:/sbin/nologin
cjs:x:1004:1004:/:/home/cjs:/sbin/nologin
[root@theon ~]# sed -n "1p" /etc/passwd
root:x:0:0:root:/root:/bin/bash
[root@theon ~]#
```

- 在第二行之后插入新行

```
sed '2a hello' /etc/passwd # 不会改变文件
sed -i '2a hello' /etc/passwd # 改变文件内容
```

```
[root@theon ~]# sed -n '2a hello' /etc/passwd
hello
[root@theon ~]# sed -in '2a hello' /etc/passwd
[root@theon ~]# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
hello
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
operator:x:11:0:operator:/root:/sbin/nologin
games:x:12:100:games:/usr/games:/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
nobody:x:99:99:Nobody:/:/sbin/nologin
systemd-network:x:192:192:systemd Network Management:/:/sbin/nologin
dbus:x:81:81:System message bus:/:/sbin/nologin
polkitd:x:999:998:User for polkitd:/:/sbin/nologin
sshd:x:74:74:Privilege-separated SSH:/var/empty/sshd:/sbin/nologin
postfix:x:89:89:/:/var/spool/postfix:/sbin/nologin
wsder:x:1000:1000:/:/home/wsder:/bin/bash
nginx:x:998:996:Nginx web server:/var/lib/nginx:/sbin/nologin
theon:x:1001:1001:/:/home/theon:/bin/bash
abc:x:1002:1002:/:/home/abc:/bin/bash
```

- 字符串替换

默认用法，`sed 's/旧字符串/新字符串/g' 文件名` 和vim的字符串替换类似

```
#在文件中查找abc 并替换成 abcer 默认会输出替换效果，但是文件不会生效
sed 's/abc/abcer/g' passwd_demo
#在文件中查找abc 并替换成 abcer 不会输出替换效果，文件会生效
sed -i 's/abc/abcer/g' passwd_demo
```

(1.4) 字符串分析

1.4.1 sort排序

排序命令，默认是按照字符顺序进行排序，可以添加参数按照数字进行排序

命令格式

```
sort [选项] 文件名（文本内容）
```

选项：

-f: 忽略大小写

-n: 以数值型进行排序，默认使用字符串型排序

-r: 降序排序

-t: 指定分隔符，默认是分隔符是制表符

-k n[,m]: 按照指定的字段范围排序。从第n字段开始，m字段结束(默认到行尾)

范例

```
cat b.txt |sort
```

```
[root@theon ~]# cat b.txt
d
c
b
a
c
a
b
d
d
d
d
c
a
d
v
b
[root@theon ~]# cat b.txt |sort
a
a
a
b
b
b
c
c
c
d
d
d
d
d
v
```

1.4.2 uniq去重

去掉相邻重复的行

命令格式

`uniq [选项] 文件名 (文本内容)`

选项:

- c: 统计重复次数并添加到输出结果
- d: 只显示重复的行
- i: 忽略大小写
- f: 避免比较前N个字段

Q: 通常情况下, 文档中重复的内容不一定是相邻的, 如何能让它们先相邻, 然后使用uniq进行去重呢?

范例

```
cat b.txt |sort |uniq
cat b.txt |sort |uniq -c
cat b.txt |sort |uniq -c|sort -nr
```

```
[root@theon ~]# cat b.txt |sort |uniq
a
b
c
d
v
[root@theon ~]# cat b.txt |sort |uniq -c
      3 a
      3 b
      3 c
      5 d
      1 v
```

```
[root@theon ~]# cat b.txt |sort |uniq -c|sort -nr
      5 d
      3 c
      3 b
      3 a
      1 v
[root@theon ~]#
```

1.4.3 wc字符统计

对字符或字符串进行统计

命令格式

wc [选项] 文件名`

选项:

-l: 只统计行数

-w: 只统计单词数

-m: 只统计字符数

范例

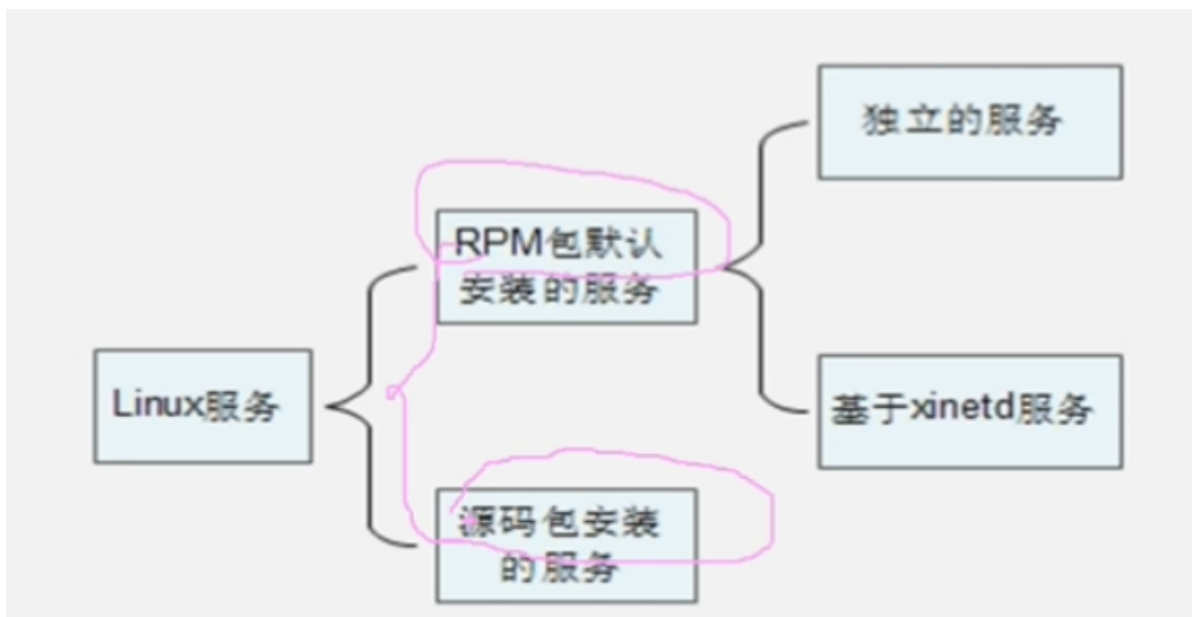

```
# 使用wc判断目录下是否有文件/目录
((($ (ls /home |wc -l) != 0)) && echo "目录下有文件或子目录" || echo "目录下啥文件目录都没有")
# if (($ (ls /home |wc -l) != 0));then echo "目录下有文件或子目录" ;else echo "目录下啥文件目录都没有" ;fi
# 粗略去统计日志文件的记录数
cat /var/log/secure|wc -l
```

二、服务管理

(2.1) 服务简介和分类

2.1.1 服务分类

按照安装方式区分服务类型



2.1.2 启动与自启动

- **服务启动:**

就是在当前系统中让服务运行，并提供功能。

- **服务自启动:**

自启动是指让服务在系统开机或重新启动之后，随着系统的启动而自动启动服务。

查询自启动的服务（只能查看RPM包安装的服务）

```
# < centos7
chkconfig --list
# >= centos7
systemctl list-unit-files --type=service|grep enabled
```

```
[root@theon ~]# systemctl list-unit-files --type=service|grep enabled
auditd.service                enabled
autovt@.service               enabled
crond.service                 enabled
dbus-org.fedoraproject.FirewallD1.service enabled
dbus-org.freedesktop.NetworkManager.service enabled
dbus-org.freedesktop.nm-dispatcher.service enabled
firewalld.service             enabled
getty@.service                enabled
irqbalance.service           enabled
kdump.service                 enabled
lvm2-monitor.service          enabled
microcode.service             enabled
NetworkManager-dispatcher.service enabled
NetworkManager-wait-online.service enabled
NetworkManager.service       enabled
postfix.service               enabled
rhel-autorelabel.service      enabled
rhel-configure.service        enabled
rhel-dmesg.service            enabled
rhel-domainname.service       enabled
rhel-import-state.service     enabled
rhel-loadmodules.service      enabled
rhel-readonly.service         enabled
rsyslog.service               enabled
sshd.service                  enabled
systemd-readahead-collect.service enabled
systemd-readahead-drop.service enabled
systemd-readahead-replay.service enabled
tuned.service                 enabled
vgauthd.service               enabled
vmtoolsd.service              enabled
```

(2.2) RPM服务的管理

2.2.1 服务相关文件

RPM安装的服务相关文件的默认位置如下：

- 启动脚本位置： `/usr/lib/systemd/system/` 启动脚本又叫Systemd 单元文件，后缀是 `.service`
- 配置文件位置： `/etc`
- 服务器产生的数据存放位置： `/var/lib`
- 服务日志位置： `/var/log`

2.2.2 启动和自启动

启动

RPM安装的服务，默认使用systemctl进行启动等相关管理，例如：

```
systemctl start|stop|restart|status nginx
```

自启动

自启动也是使用systemctl进行管理，例如：

```
systemctl enable|disable nginx
```

(2.3) 源码包服务管理

2.3.1 服务相关文件

对于 Linux 中源码包安装的服务，**没有一个固定的、默认的存储位置**，部分文件的相关信息可能在 README或INSTALL文件中有描述

```
Apache-apr2.dsw  BuildAll.dsp  CMakeLists.txt  emacs-style  include  libht
Apache.dsw       BuildBin.dsp  config.layout   httpd.dep    INSTALL  libht
apache_probes.d  buildconf    configure       httpd.dsp    InstallBin.dsp  LICEN
[root@theon httpd-2.4.63]# cat INSTALL

APACHE INSTALLATION OVERVIEW

Quick Start - Unix
-----

For complete installation documentation, see [ht]docs/manual/install.html or
http://httpd.apache.org/docs/2.4/install.html

$ ./configure --prefix=PREFIX
$ make
$ make install
$ PREFIX/bin/apachectl start

NOTES: * Replace PREFIX with the filesystem path under which
        Apache should be installed. A typical installation
        might use "/usr/local/apache2" for PREFIX (without the
```

- 启动脚本位置

利用源码软件安装根目录（例如 `/usr/local/<servicename>`）下 `bin` 目录中的二进制命令，自行编写一个启动脚本，然后加载到 `systemctl` 中统一管理，例如下面启动脚本

```
#管理员或用户自定义 systemd 单元文件 的首选和推荐位置/etc/systemd/system
[root@theon ~]# vim /etc/systemd/system/httpd.service

[Unit]
Description=The Apache HTTP Server
After=network.target remote-fs.target nss-lookup.target

[Service]
Type=forking
PIDFile=/usr/local/apache2/logs/httpd.pid
ExecStart=/usr/local/apache2/bin/apachectl start
ExecReload=/usr/local/apache2/bin/apachectl graceful
ExecStop=/usr/local/apache2/bin/apachectl stop
PrivateTmp=true

[Install]
WantedBy=multi-user.target

# 将编写的启动脚本加载到systemctl中
[root@theon ~]# systemctl daemon-reload
```

- 配置文件位置

可能在源码安装目录中有配置文件demo文件，也有可能在安装完成后，在源码软件安装根目录（例如 `/usr/local/<servicename>`）下 `config` 目录中，例如 `/usr/local/apache2/conf/httpd.conf`

- 数据存储文件

在源码软件安装根目录 `/usr/local/<servicename>` 下查找

- 日志文件

通常在在源码软件安装根目录 `/usr/local/<servicename>` 下 `logs` 目录中

关于服务相关文件的总结：

1、安装之前查看其安装目录下的 `README`、`INSTALL` 或 `docs` 文件夹，这些文档通常会详细说明默认的文件路径和基本的配置指南。

2、启动脚本（systemd单元文件）需要手动编写

3、尽量多找一下源码软件安装根目录 `/usr/local/<servicename>`

2.3.2 启动和自启动

启动

前面已经介绍过，自写systemd单元文件，加载到systemd后，使用systemctl管理启动

自启动

利用systemctl 或者 写入到 `/etc/rc.local` 文件中：

```
echo "/usr/local/apache2/bin/apachectl start" >> /etc/rc.local
chmod +x /etc/rc.local
```

扩展：关于文件/etc/rc.local

是文件/etc/rc.d/rc.local的软链接，早期linux操作系统串行启动方式（SysV-init）中，在系统完成所有标准启动服务（如网络、文件系统等）之后，会自动执行这个文件，是系统启动的倒数第二步（最后一步是加载登录页面）。

串行启动，是将每个系统服务进行编号，然后按照编号依次进行启动

当前使用systemd并行启动后，rc.local文件其实已经被废弃。但是Systemd 通常会提供一个 `rc-local.service` 兼容性单元，如果 `/etc/rc.local` 存在并且可执行，Systemd 仍然会调用它，

并行启动是在systemd单元文件中写好服务启动依赖，然后按照依赖同时去启动服务

作业

一、登录记录统计

任务描述：请编写一个shell脚本来分析文件secure，找到登录失败次数最多的ip和该IP的失败次数，以及该IP失败次数和总登录失败次数的占比（保留两位小数）

提示：awk 'BEGIN {printf "%.2f%",(1 / 3) * 100}'

任务提交：脚本执行后的输出截图，如下范例图

```
[root@theon secure_anaylist]# sh work_20250605.sh
```

```
secure 分析结果如下:
```

```
登录失败次数最多的ip为 74.23.156.19 ,该IP登录失败次数为 7547 ,在登录失败中占比50.01%
```

```
[root@theon secure_anaylist]#
```